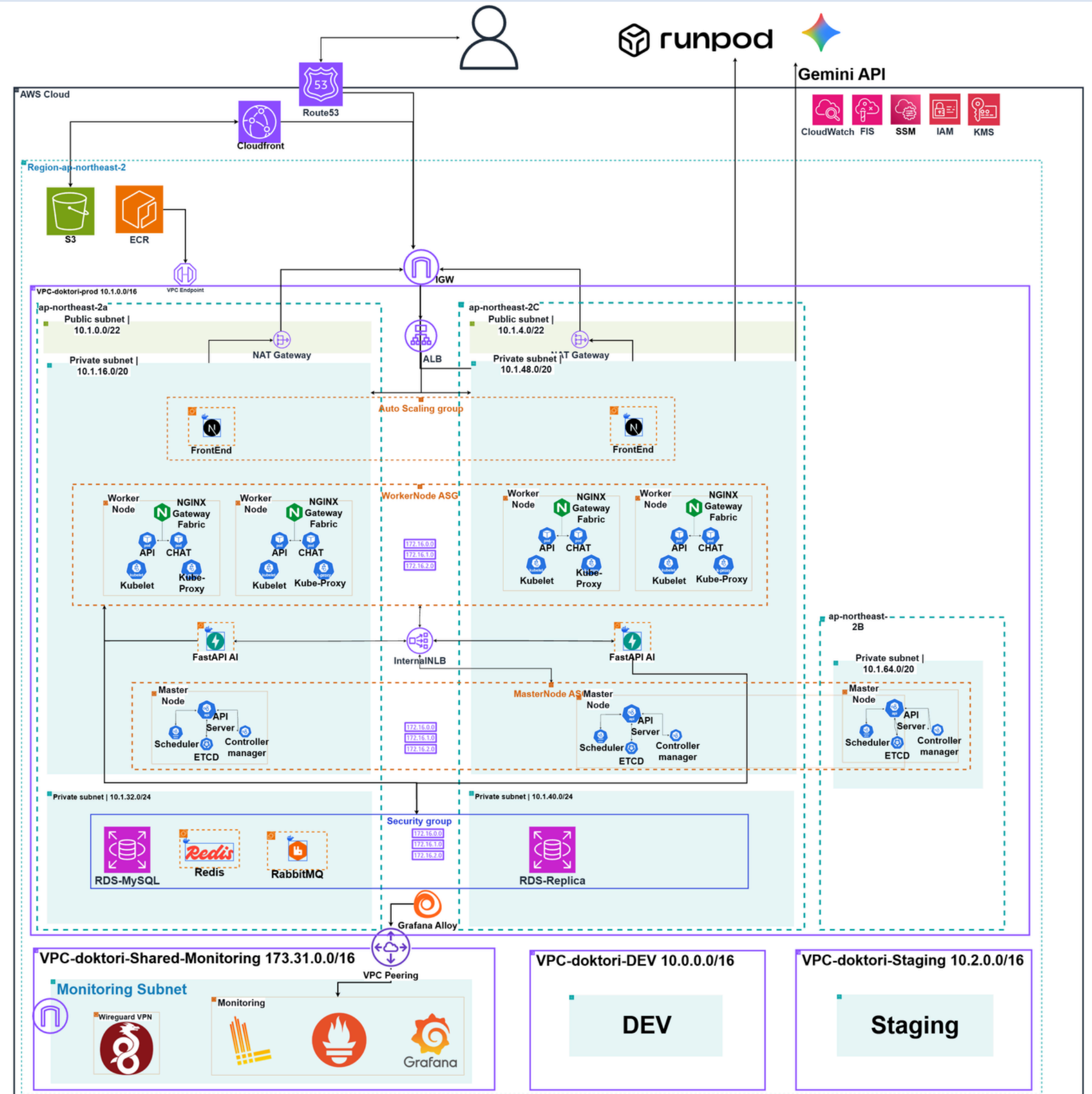
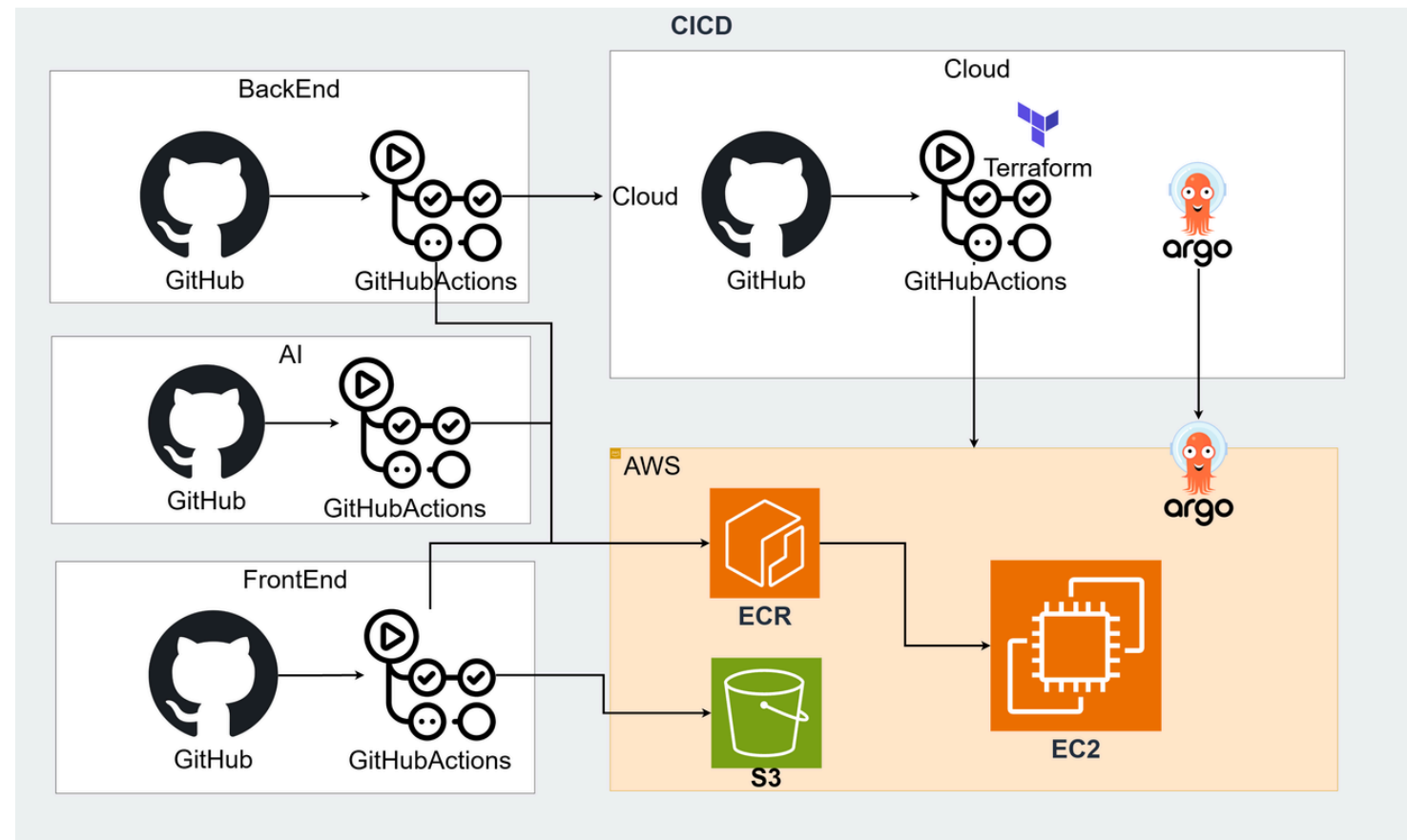


Portfolio

안정적인 서비스를 만드는 Engineer, 김정빈입니다.

배포 파이프라인과 아키텍처



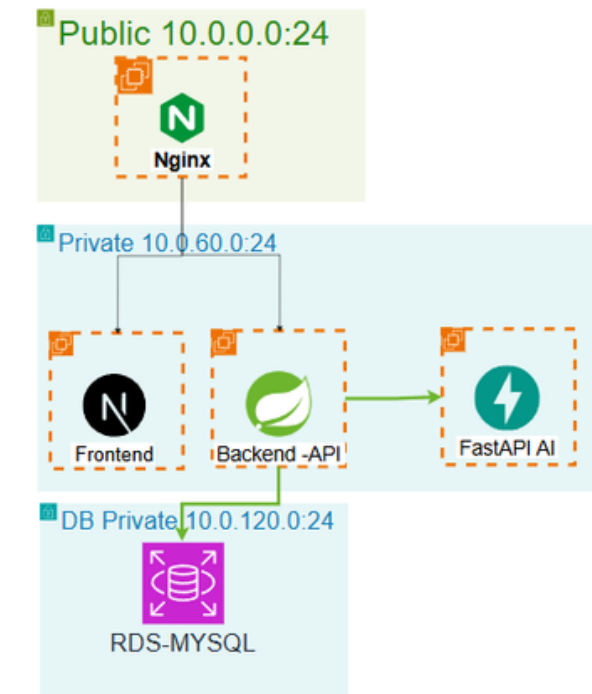
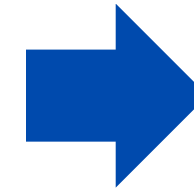
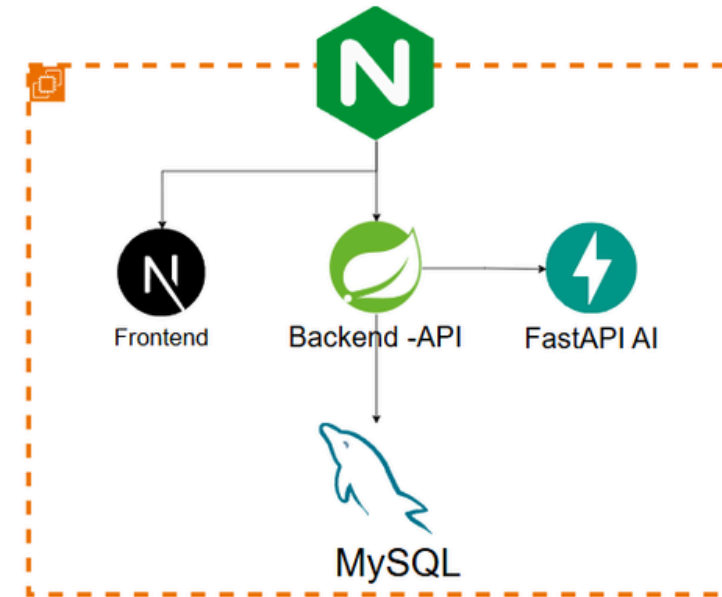
Migration

DB 마이그레이션을 위해 Kafka나 AWS DMS 같은 무거운 도구를 도입하는 것이 최선의 선택일까?

마이그레이션 스크립트

문제 상황 : 운영 DB를 단일 EC2에서 RDS로 전환. 단순 중지 방식은 mysqldump + import 전체(약 30분) 다운타임.

목표 : 단일 인스턴스 환경 분리, 다운 타임 최소화



- 워크로드 분석: 스케줄러 2개와 AI 배치의 실행 시간을 피해 컷오버를 진행. 전환 중 사용자 쓰기 요청은 일시적으로 실패할 수 있으나, 프론트에 입력값을 유지해 사용자가 재제출할 수 있도록 대응.
- 비용/복잡도: Kafka, DMS 인프라를 추가하는 건 운영 부담

컴포넌트	쓰기 빈도	컷오버 처리
사용자 액션 (독후감 제출, 모임 생성 등)	상시	프론트가 입력 데이터 보유 → 5xx 시 재전송
MeetingScheduler	매일 00:00	컷오버 시점 회피
NotificationScheduler	매시 :00, :30	컷오버 시점 회피
AI 배치	주 1회	컷오버 시점 회피

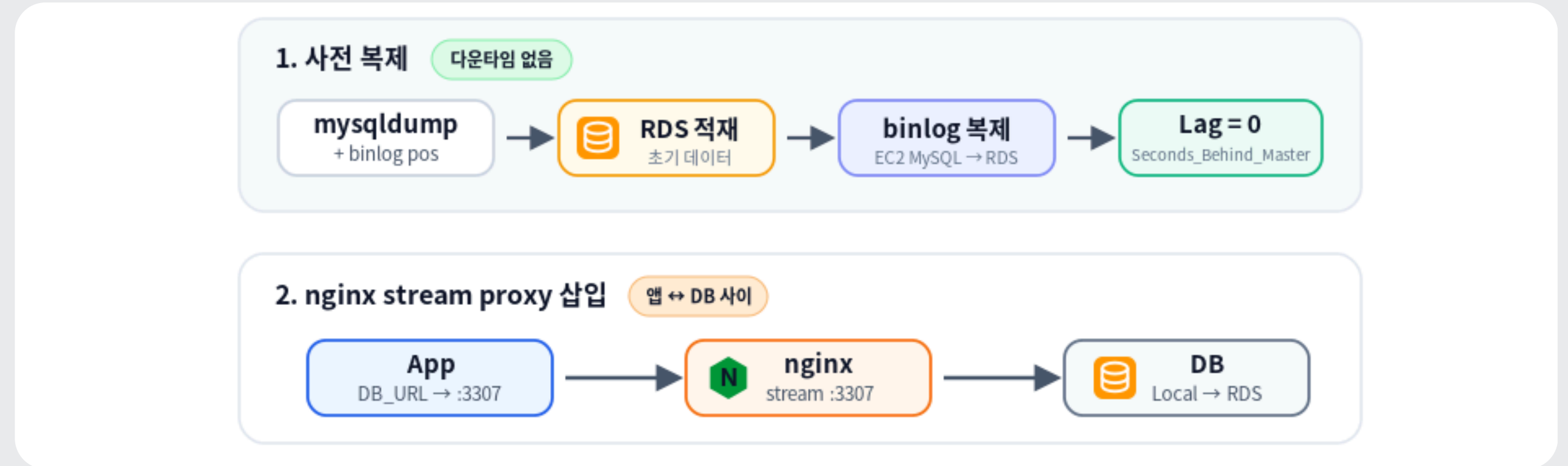
Migration

DB 마이그레이션을 위해
Kafka나 AWS DMS 같은
무거운 도구를 도입하는 것이
최선의 선택일까?

마이그레이션 스크립트

채택 방식 :

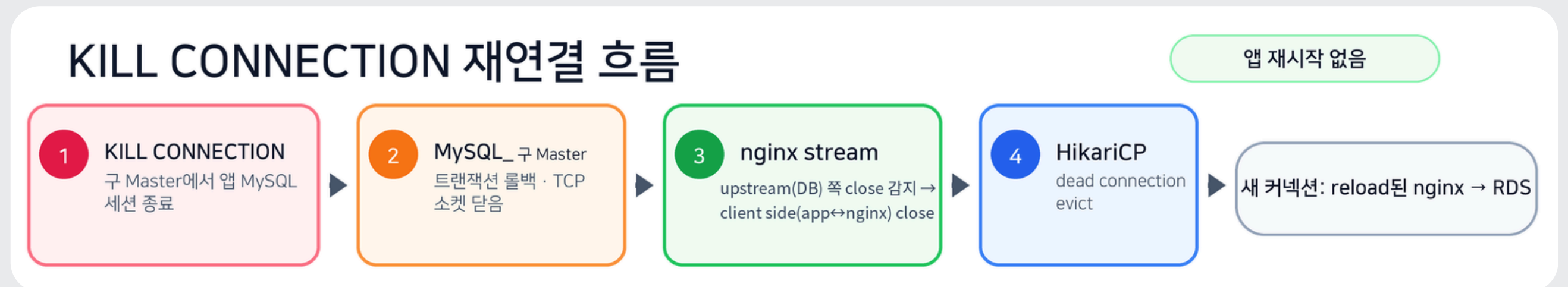
- 사전 단계: binlog M-S 복제로 RDS를 실시간 동기화 상태로 유지
- 컷오버 단계: nginx stream proxy upstream 교체로 앱 재시작 없이 경로 전환



문제 1 : HikariCP 커넥션 eviction

- nginx reload 후 upstream → RDS 이나 HikariCP 커넥션이 기존 로컬 MySQL을 계속 사용.
 - nginx reload는 기존 worker 프로세스를 살려두므로 기존 TCP 커넥션은 경로 변경 없이 그대로 유지됨.
- connection-test-query 시도: `SELECT IF(@@global.read_only=0, 1, CAST('readonly' AS UNSIGNED))`로 read_only 상태를 감지해 커넥션 evict 시도. 동작은 하나, JDBC4 isValid() 대신 매 health check마다 SQL을 실행하는 오버헤드 존재

해결 : KILL CONNECTION



Migration

DB 마이그레이션을 위해 Kafka나 AWS DMS 같은 무거운 도구를 도입하는 것이 최선의 선택일까?

마이그레이션 스크립트

컷오버 : 모든 과정을 스크립트화하여 진행



정합성 보장 : super_read_only 설정 후 binlog File·Position 일치 확인 뒤 승격 → 구 DB 커밋 트랜잭션 전량 RDS 반영

롤백 : nginx stream conf upstream을 로컬 MySQL로 되돌리고 reload → 즉시 원복

핵심 성과:

항목	mysqldump 방식	nginx stream proxy
서비스 다운타임	~30분	0
쓰기 불가 구간	~30분	~5초
앱 코드 변경	DB URL + 재배포	없음
롤백	dump 시점 복원	nginx reload 즉시

약 360배 단축

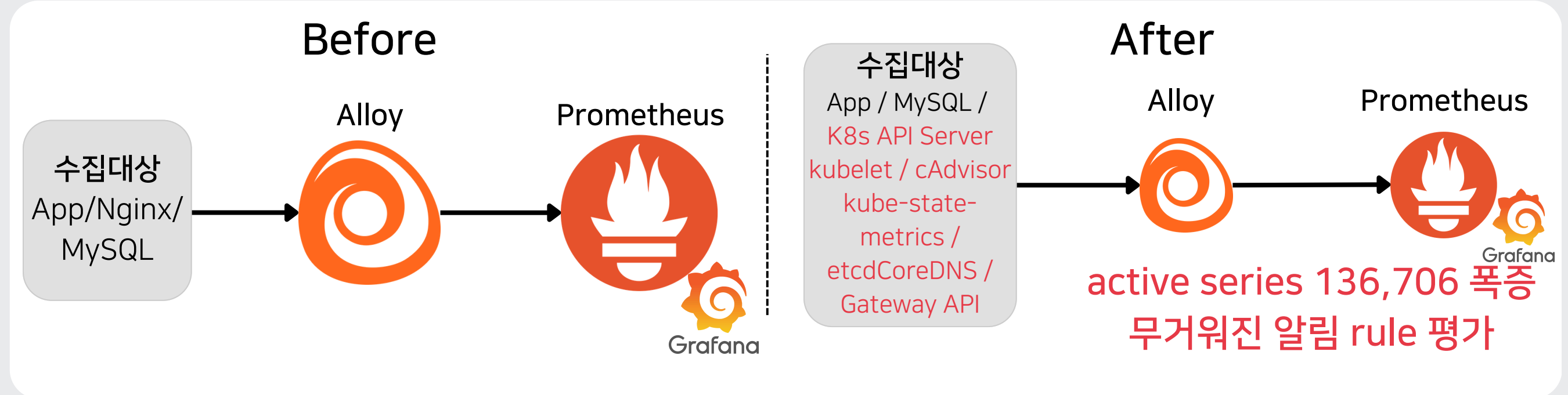


Monitoring

폭증하는 메트릭을 감당하기 위해 무작정 모니터링 서버의 스펙을 올리는 것(Scale-up)이 근본적인 해결책일까?

모니터링/알림 구성링크

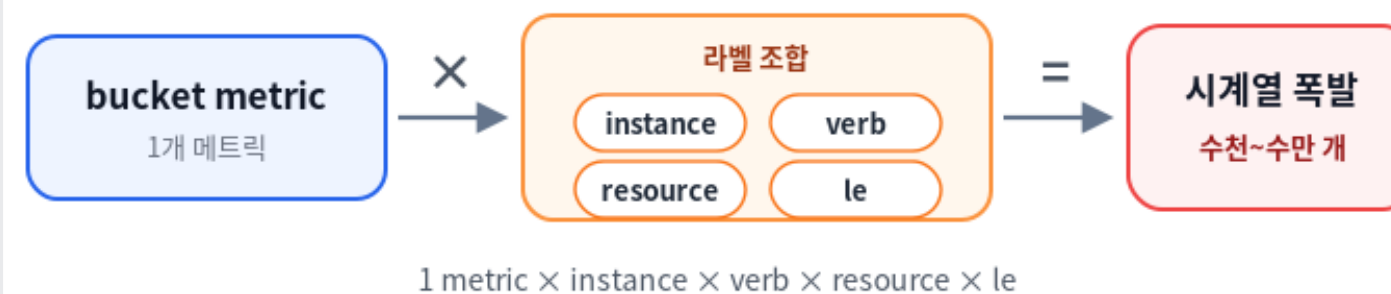
문제 상황 : 쿠버네티스 도입 이후, 메트릭 증가에 따른 기존 모니터링 서버의 리소스 부족 발생



원인 분석 :

- TSDB는 메트릭 이름이 아니라 각 라벨 조합을 시계열로 저장 → 메트릭 수가 적어도 라벨이 곱해지면 시계열은 폭발
- label set 기준, 약 47,000개, 전체의 35% 수준이 histogram bucket 계열, bucket 메트릭은 instance, verb, resource, le 같은 라벨이 조합 → 고카디널리티 메트릭 다수 존재

bucket metric X 라벨 조합 = 시계열 폭발



고카디널리티 메트릭 분류표

Metric family	시계열 수
apiserver 미사용 bucket 계열	~20,000
etcd_request_duration_seconds_bucket	10,152
kubelet_runtime_operations_duration_seconds_bucket	1,965
container_tasks_state	2,100
apiserver_request_duration_seconds_bucket	13,032

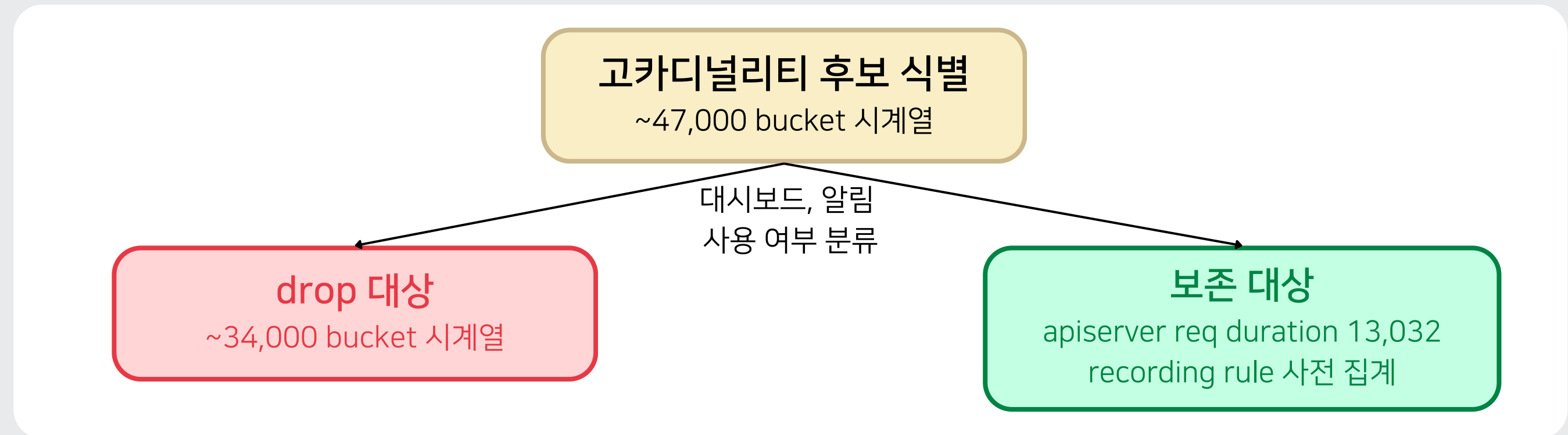
Monitoring

폭증하는 메트릭을 감당하기 위해 무작정 모니터링 서버의 스펙을 올리는 것(Scale-up)이 근본적인 해결책일까?

모니터링/알림 구성링크

판단&해결 :

- 단순 모니터링 서버 리소스 향상시키지 않고 필요하지 않은 histogram bucket 을 제거 하여 리소스 절약 우선



- recording rule : 13,032 raw 스캔 → ~50 조회



성과 :

- 미사용 고카디널리티 시계열 약 34,000개 이상 수집 차단, recording rule을 통한 모니터링 컴퓨팅 절약
- 모니터링 서버 비용 50% 이상 절감

Alert

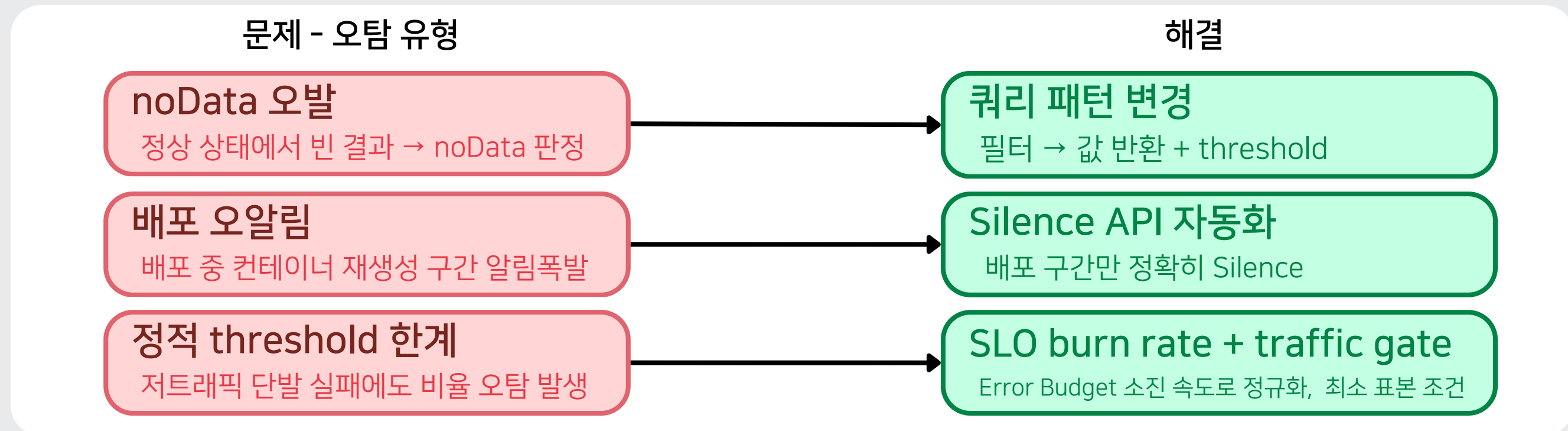
오탐 알림의 피로도를 어떻게 줄일 수 있을까?

단순 에러율(%) 기반의 정적 임계치(Static Threshold)가 트래픽이 적은 새벽 시간대에도 유효할까?

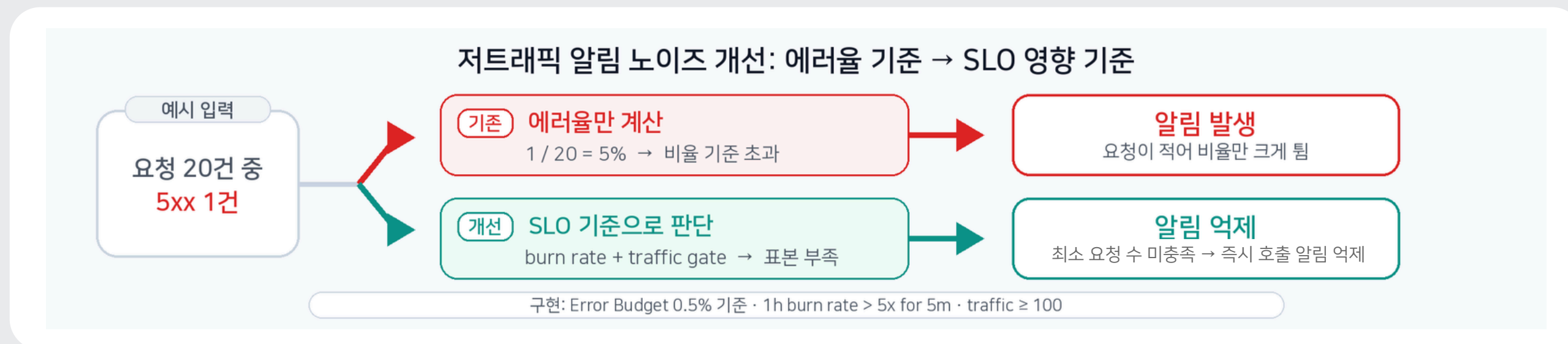
모니터링/알림 구성링크

문제 상황 : 알림이 실제 장애가 아닌 상황(배포, 정상 상태, threshold 진동)에서 반복 발화.
알림 신뢰도 하락, 오탐으로 인한 알림 피로도 증가

오탐 - 해결 :



burn rate :



성과 :

- 알림 오탐 약 60% 이상 감소

CI/CD with GitHub Actions

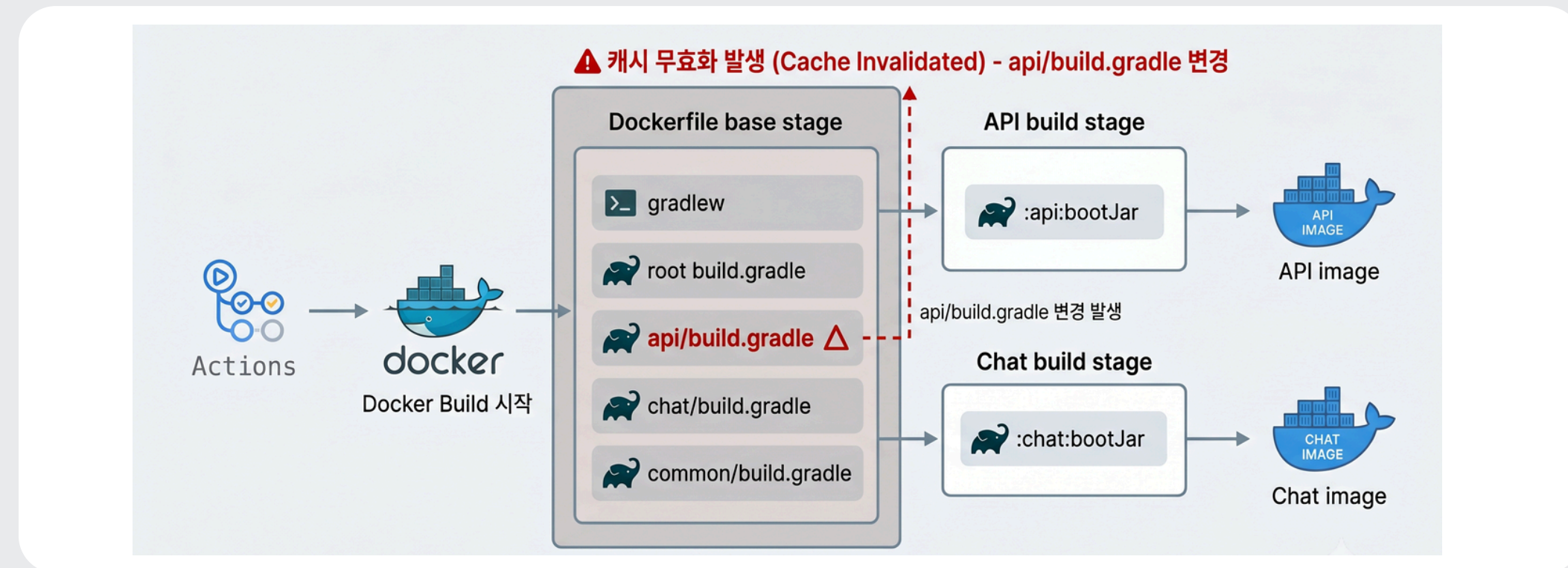
증가한 빌드 시간 줄일 수 있을까?

캐시가 의도한 대로 잘 사용되고 있는 것일까?

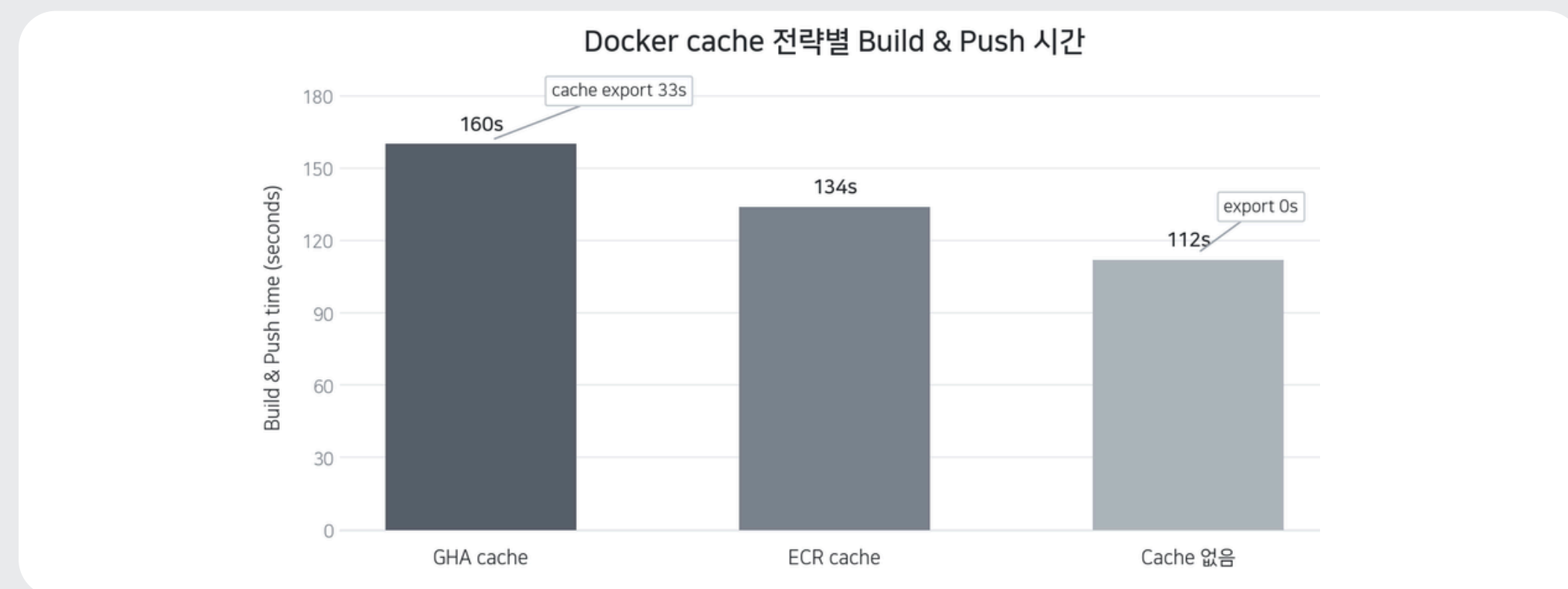
트러블 슈팅 기록링크

문제 상황 : 백엔드 멀티모듈 분리 이후 CI/CD 배포 시간이 2배 이상 증가

원인 분석 1 : 단일 모듈 변경에도 전체 이미지 빌드 발생



원인 분석 2 : ephemeral GitHub Actions runner에서는 cache export/import I/O가 더 큼



CI/CD with GitHub Actions

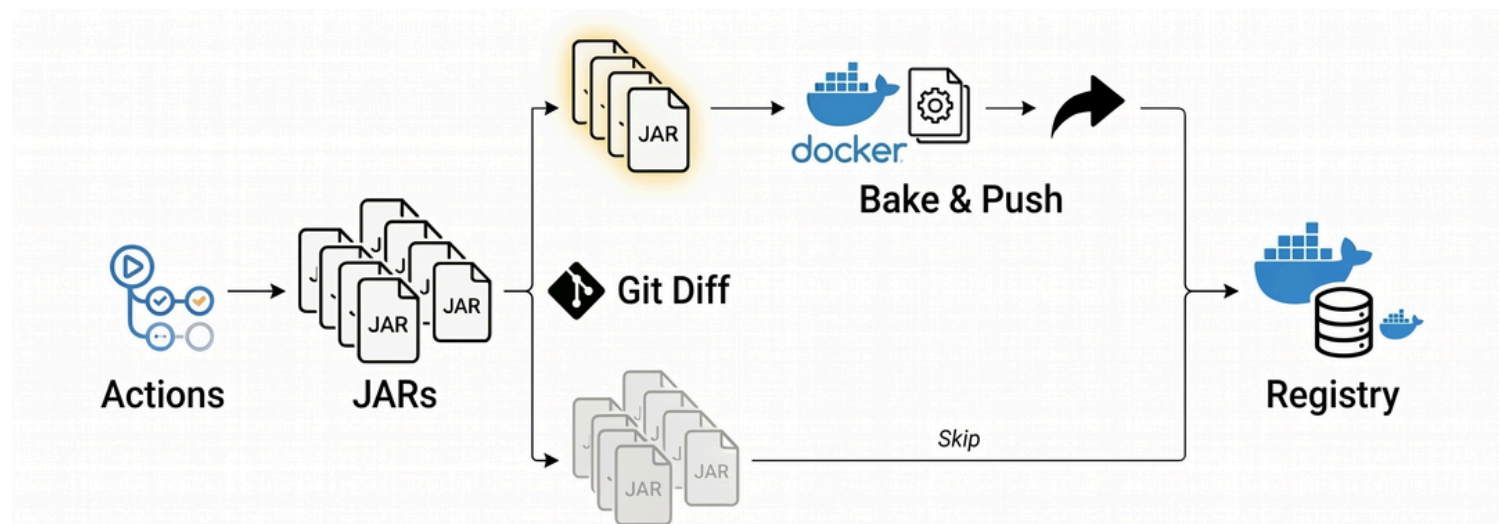
증가한 빌드 시간 줄일 수 있을까?

캐시가 의도한 대로 잘 사용되고 있는 것일까?

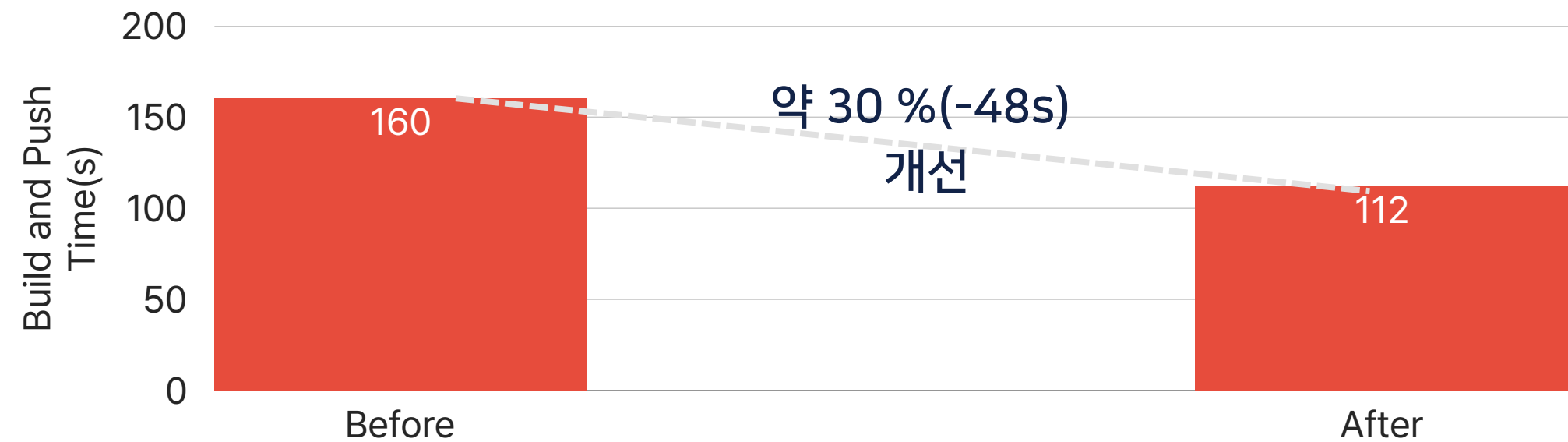
트러블 슈팅 기록링크

해결 과정 :

1. Gradle 빌드를 CI 단계로 분리하여 증분 빌드·dependency cache 활용, Docker Bake 타겟을 모듈별로 분리하고 Git diff 기반 변경 감지로 필요한 타겟만 빌드·푸시 → 모듈 간 빌드 독립성 확보 및 불필요한 빌드 시간 제거
2. ephemeral runner 환경에서 캐시 I/O 비용(export 33s)이 로컬 재빌드 비용(5s)을 초과 → Docker 캐시 제거



핵심 성과: Build and Push 구간 30% 개선 (2m 40s → 1m 52s), 모듈 간 변경 영향 범위 축소



Infrastructure as Code

"수동 조작과 코드(IaC) 간의
상태 불일치(Drift)로 인한
장애를 어떻게 사전에 차단할
수 있을까?"

IaC 구성 링크

■ 배경 상황 : 인프라 콘솔 관리 중 휴먼에러 누적, 중지 서버 삭제 사고 계기로 IaC 도입 결정

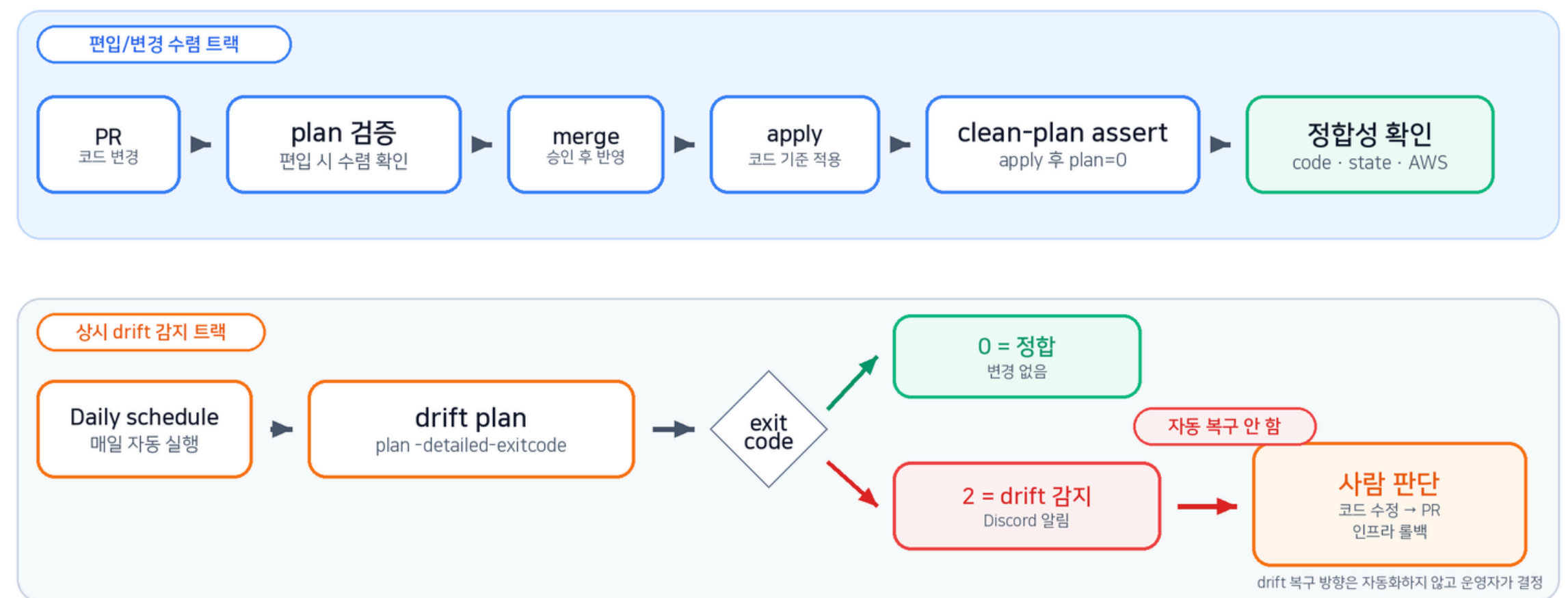
■ 문제 상황 1 : AWS - Code - State 불일치

■ 분석 1 :

- 초기 IaC 과정에서의 불일치 → import 필요
- 운영 중 발생하는 drift → 팀원과의 합의, 감지 메커니즘 필요

■ 해결 :

- PR plan 검증(편입 시 수렴)
- scheduled drift 감지(매일 plan -detailed-exitcode → Discord) → apply 후 clean-plan 수렴 검증.
drift는 자동 복구 안 함 — 코드 수정 vs 인프라 롤백은 사람 판단으로 남긴 의도적 설계.



■ 성과 :

- drift 처리로 인한 시간 낭비 개선(도입 이전 대비 30%이상 개선)

Infrastructure as Code

Blast Radius 을 어떻게
분리하고 최소화할 수 있을까?

각 리소스의 서로 다른 변경
주기와 의존성을 엉킴 없이
관리하는 최적의 구조는
무엇일까?

laC 구성 링크

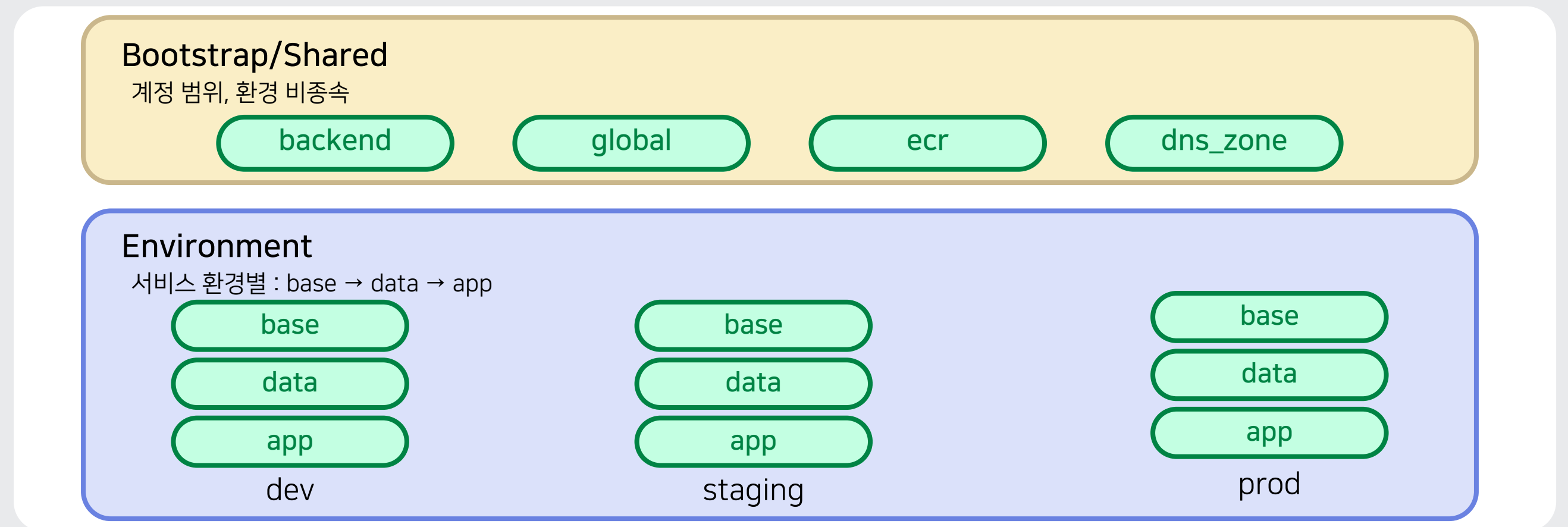
문제 상황 2 : 단일 state → (1) 필요한 단위만 끄고 내릴 수 없고 (2) 실수 영향이 전체에 미침.

분석 : state 분리 작업 필요

- AWS 리소스 타입별 분리 → 서비스 하나를 변경하는 일이 여러 state에 흩어짐
- 분리 기준을 함께 변하고 무엇에 의존하느냐(역할)로 변경 → 변경 단위와 state 경계 일치 시킴

해결 :

- state를 역할·의존성 기준(base → data → app)으로 재편. 변경 단위와 경계가 일치해 순서만 따라가면 됨. 의존성은 remote_state로 명시, 순서는 CI의 needs로 표현 및 문서화.



IaC 를 통한 성과 : 인프라 프로비저닝 시간 약 90% 감소(4시간 → 25분), 휴먼에러 감소

개요

- ✓ 기간: 3개월
- ✓ 역할: Cloud 운영/배포 담당

핵심기여

- ✓ AWS 인프라 설계 및 구축, 운영
- ✓ 모니터링/알림 시스템
- ✓ CI/CD 파이프라인 구축

주요목표

- ✓ 시공간과 진입장벽의 제약을 극복한 독서 토론 플랫폼

WHY & HOW

✓ Pain Point 1

시공간적 제약

오프라인 중심 운영
시간·장소 맞춤 필요

✓ Pain Point 2

품질 편차

모임별 운영 방식 상이
토론 주제·진행 수준의 편차

✓ Pain Point 3

진입 장벽

초심자 참여 부담
독서 모임에 대한 심리적 거리감

✓ Solution 1

온라인 전환

시공간 제약을 줄여
언제 어디서나 참여 가능

✓ Solution 2

AI 보조 기능

AI를 사용하여
토론 품질 보완 기능 제공

✓ Solution 3

접근성 강화

가벼운 채팅 토론방 운영을 통한
심리적 진입 장벽 완화

독토리



독서 토론 마을 - 프로젝트 소개

부담없이 즐기는 채팅 토론방

모임 페이지

